

# Boundary Conditions

## Intro And Scope

This document explains what boundary conditions are in the context of polynomial trajectory planning and lists the boundary conditions relevant to each polynomial degree currently in scope for the project: degree 3, 5, 7, and 9.

The goal of the document is educational before technical. It clarifies:

- what a boundary condition actually is;
- why the number of boundary conditions depends on the polynomial degree;
- how to distinguish necessary, possible, and redundant conditions;
- how the 1D reasoning extends to 3D.

This document covers the learning part of Requirement 3 in 01-requirements.md.

## Symbols Used

The notation follows SymbolsAndNomenclature.md. The main symbols used here are:

- $t_0$ : initial time;
- $t_f$ : final time;
- $T = t_f - t_0$ : motion duration;
- $x(t)$ : scalar position law in 1D;
- $\mathcal{S}_0, \mathcal{S}_f$ : initial and final state containers when endpoint data are grouped compactly;
- $x_0, x_f$ : initial and final position;
- $v_0, v_f$ : initial and final velocity;
- $a_0, a_f$ : initial and final acceleration;
- $j_0, j_f$ : initial and final jerk;
- $s_0, s_f$ : initial and final snap.

When needed, the generic  $k$ -th derivative is written as  $x^{(k)}(t)$ .

## What A Boundary Condition Is

A boundary condition is a prescribed value imposed on the trajectory or on one of its derivatives at a specific time, usually at the beginning or at the end of a segment.

Typical examples are:

$$x(t_0) = x_0, \quad x(t_f) = x_f$$

for position, or:

$$\dot{x}(t_0) = v_0, \quad \dot{x}(t_f) = v_f$$

for velocity.

So, in simple terms:

- a polynomial provides unknown coefficients;
- boundary conditions provide equations;

- when the number of independent equations matches the number of unknown coefficients, the polynomial can be determined uniquely.

This is the key idea behind the whole document.

## Why The Number Of Boundary Conditions Depends On Degree

A polynomial of degree  $n$  has  $n + 1$  coefficients:

$$x(t) = a_0 + a_1t + a_2t^2 + \dots + a_nt^n$$

The unknowns are therefore:

$$a_0, a_1, a_2, \dots, a_n$$

which are  $n + 1$  scalar unknowns.

If the duration  $T$  is already known and fixed, then the standard coefficient-determination problem requires:

$$n + 1$$

independent scalar boundary conditions.

This immediately gives the count for the degrees currently in scope:

- degree 3: 4 conditions;
- degree 5: 6 conditions;
- degree 7: 8 conditions;
- degree 9: 10 conditions.

## The Natural Symmetric Endpoint Pattern

For the odd degrees used in this project, there is a natural symmetric pattern based on matching the same derivative orders at the start and at the end of the segment.

The pattern is:

- degree 3: position and velocity at both ends;
- degree 5: position, velocity, and acceleration at both ends;
- degree 7: position, velocity, acceleration, and jerk at both ends;
- degree 9: position, velocity, acceleration, jerk, and snap at both ends.

This pattern is natural because it gives exactly the required number of scalar conditions:

$$2 \times \frac{n + 1}{2} = n + 1$$

for odd degree  $n$ .

This is not the only possible formulation, but it is the most direct and educational one for this project.

## Three Condition Classes

To keep the terminology consistent across the project, boundary conditions are divided into three classes.

### Necessary Conditions

Necessary conditions are the conditions that must be specified in the chosen formulation to determine the polynomial uniquely.

In this document, the default formulation is:

- fixed polynomial degree;
- fixed start and end times;
- endpoint conditions only;
- symmetric use of derivative orders at the two endpoints.

Under this formulation, the necessary conditions are the ones listed degree by degree in the next sections.

### Possible Conditions

Possible conditions are conditions that can be used in broader or alternative formulations, but are not required in the default formulation.

Examples:

- replacing one endpoint derivative condition with another independent condition;
- prescribing an interior-point condition;
- using higher-order derivative data while freeing another parameter such as duration;
- using a non-symmetric formulation between start and end conditions.

So “possible” does not mean “needed here”. It means “admissible in some extended formulation”.

### Redundant Conditions

Redundant conditions are extra independent conditions beyond what the fixed-degree, fixed-duration polynomial can satisfy uniquely.

If a degree- $n$  polynomial with fixed duration already has  $n + 1$  independent scalar conditions, then adding another independent condition makes the problem overconstrained.

In that situation, one of the following happens:

- the system has no exact solution;
- one of the conditions is not actually independent;
- the formulation must be changed, for example by increasing degree or adding free parameters.

This is why the distinction between possible and redundant is important:

- possible means usable in some reformulated problem;
- redundant means extra within the current fixed formulation.

## Condition Catalogue In 1D

Before listing the degrees one by one, it is useful to define the basic catalogue of endpoint conditions in 1D:

$$x_0, x_f, v_0, v_f, a_0, a_f, \dot{j}_0, \dot{j}_f, s_0, s_f$$

These correspond to:

- position at start and end;
- velocity at start and end;
- acceleration at start and end;
- jerk at start and end;
- snap at start and end.

For the degrees currently in scope, this catalogue is sufficient for the main endpoint-based theory.

When a full endpoint condition set is treated as a compact object, the state notation from `SymbolsAndNomenclature.md` may also be used. For example, in the degree-9 case:

$$\mathcal{S}_0 = (x_0, v_0, a_0, \dot{j}_0, s_0), \quad \mathcal{S}_f = (x_f, v_f, a_f, \dot{j}_f, s_f)$$

This notation is particularly useful when discussing complete endpoint data sets without overloading the snap symbol  $s$ .

## Degree 3

### Polynomial Structure

A cubic polynomial has four coefficients:

$$x(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$$

Therefore it needs four independent scalar conditions in the default formulation.

### Necessary Conditions

The natural necessary set is:

$$x_0, x_f, v_0, v_f$$

This means:

$$x(t_0) = x_0, \quad x(t_f) = x_f, \quad \dot{x}(t_0) = v_0, \quad \dot{x}(t_f) = v_f$$

## Possible Conditions

Possible conditions in broader formulations may include:

- acceleration values at one or both endpoints;
- jerk values at one or both endpoints;
- an interior position value;
- unknown duration combined with a different choice of endpoint conditions.

These conditions are possible only if the full formulation is changed so that the total number of independent unknowns and equations remains balanced.

## Redundant Conditions

In the default cubic formulation with fixed duration, the following types of additions are redundant:

- prescribing  $a_0$  or  $a_f$  in addition to  $x_0, x_f, v_0, v_f$ ;
- prescribing  $j_0$  or  $j_f$  in addition to the same set;
- adding any extra independent endpoint condition beyond the original four.

So the basic educational message is simple: a cubic segment naturally controls position and velocity, not the full higher-order hierarchy.

## Degree 5

### Polynomial Structure

A quintic polynomial has six coefficients:

$$x(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5$$

Therefore it needs six independent scalar conditions in the default formulation.

### Necessary Conditions

The natural necessary set is:

$$x_0, x_f, v_0, v_f, a_0, a_f$$

This means:

$$x(t_0) = x_0, \quad x(t_f) = x_f$$

$$\dot{x}(t_0) = v_0, \quad \dot{x}(t_f) = v_f$$

$$\ddot{x}(t_0) = a_0, \quad \ddot{x}(t_f) = a_f$$

### Possible Conditions

Possible conditions in extended formulations may include:

- jerk at one or both endpoints;
- one or more interior conditions;
- duration treated as an unknown;
- a non-symmetric distribution of endpoint derivative conditions.

### Redundant Conditions

In the default quintic formulation with fixed duration, the following become redundant if added independently:

- $\dot{j}_0, \dot{j}_f$ ;
- any snap endpoint condition;
- any extra independent condition beyond the six already listed.

The quintic therefore extends the cubic case by naturally controlling acceleration at the boundaries.

## Degree 7

### Polynomial Structure

A seventh-degree polynomial has eight coefficients:

$$x(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5 + a_6t^6 + a_7t^7$$

Therefore it needs eight independent scalar conditions in the default formulation.

### Necessary Conditions

The natural necessary set is:

$$x_0, x_f, v_0, v_f, a_0, a_f, \dot{j}_0, \dot{j}_f$$

This means that position, velocity, acceleration, and jerk are prescribed at both endpoints.

### Possible Conditions

Possible conditions in broader formulations may include:

- snap at one or both endpoints;
- interior constraints;
- alternative mixes of endpoint data;
- duration or other parameters promoted to unknowns.

## Redundant Conditions

In the default seventh-degree formulation with fixed duration, the following become redundant if added independently:

- $s_0, s_f$ ;
- any extra endpoint condition beyond the eight already used;
- any other independent scalar condition that does not replace an existing one.

The educational interpretation is that the seventh-degree polynomial is the first degree in this project that naturally includes jerk at both ends.

## Degree 9

### Polynomial Structure

A ninth-degree polynomial has ten coefficients:

$$x(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5 + a_6t^6 + a_7t^7 + a_8t^8 + a_9t^9$$

Therefore it needs ten independent scalar conditions in the default formulation.

### Necessary Conditions

The natural necessary set is:

$$x_0, x_f, v_0, v_f, a_0, a_f, \dot{j}_0, \dot{j}_f, s_0, s_f$$

This means that position, velocity, acceleration, jerk, and snap are prescribed at both endpoints.

### Possible Conditions

Possible conditions in broader formulations may include:

- crackle-related information if the problem is reformulated;
- interior-point constraints;
- asymmetric endpoint specifications;
- duration included among the unknowns;
- optimization-based or approximate fitting formulations.

## Redundant Conditions

In the default ninth-degree formulation with fixed duration, the following become redundant if added independently:

- any additional crackle or pop endpoint condition;
- any extra interior scalar condition that is imposed without releasing another condition or adding unknowns;
- any extra independent scalar condition beyond the ten already used.

This is why snap has a special role in the degree-nine case: it is the highest derivative that still fits naturally into the standard symmetric endpoint formulation.

## Summary Table For 1D

The degree-by-degree classification can be summarized as follows.

| Degree | Coefficients | Necessary conditions in the default formulation                | Examples of possible extra conditions in broader formulations  | Redundant in the default fixed formulation |
|--------|--------------|----------------------------------------------------------------|----------------------------------------------------------------|--------------------------------------------|
| 3      | 4            | $x_0, x_f, v_0, v_f$                                           | $a_0, a_f$ , interior position, unknown duration               | any fifth independent condition            |
| 5      | 6            | $x_0, x_f, v_0, v_f, a_0, a_f$                                 | $\dot{j}_0, \dot{j}_f$ , interior conditions, unknown duration | any seventh independent condition          |
| 7      | 8            | $x_0, x_f, v_0, v_f, a_0, a_f, \dot{j}_0, \dot{j}_f$           | $s_0, s_f$ , interior conditions, unknown duration             | any ninth independent condition            |
| 9      | 10           | $x_0, x_f, v_0, v_f, a_0, a_f, \dot{j}_0, \dot{j}_f, s_0, s_f$ | crackle-related data, interior conditions, unknown duration    | any eleventh independent condition         |

This table should be read carefully:

- the “necessary” column refers only to the standard formulation adopted as default in this project;
- the “possible” column does not mean those conditions are added on top unchanged;
- the “redundant” column means “redundant unless the formulation itself is changed”.

## Independence Matters As Much As Count

Counting conditions is necessary, but not sufficient. The conditions must also be independent.

For example, a degree-5 polynomial needs six scalar conditions, but not every group of six conditions is equally well chosen. If two conditions are linearly dependent in the coefficient system, they do not provide six independent equations.

So the complete rule is:

- correct number of conditions;



- independence of those conditions;
- consistency with the chosen formulation.

This point will become especially important in PolynomialCoefficientDetermination.

## What Changes In 3D

The logic does not change in 3D. What changes is that the scalar 1D problem is applied to three Cartesian components.

If the three coordinates are treated independently, then each axis has its own set of scalar boundary conditions:

$$x_0, x_f, v_{x,0}, v_{x,f}, \dots$$

$$y_0, y_f, v_{y,0}, v_{y,f}, \dots$$

$$z_0, z_f, v_{z,0}, v_{z,f}, \dots$$

In compact vector notation, the corresponding endpoint conditions are:

$$\mathbf{p}_0, \mathbf{p}_f, \mathbf{v}_0, \mathbf{v}_f, \mathbf{a}_0, \mathbf{a}_f, \mathbf{j}_0, \mathbf{j}_f, \mathbf{p}_0^{(4)}, \mathbf{p}_f^{(4)}$$

The educational idea is still the same:

- first understand the scalar case;
- then apply the same rule component-wise in 3D.

The main difference in 3D is therefore not the nature of the boundary conditions, but the bookkeeping and the later interpretation of multi-axis coordination.

## Relation To The Data Model

The current data model in 02-data-model.md defines BoundaryConditions as a fixed set of ten scalar fields (startPosition/endPosition, startVelocity/endVelocity, startAcceleration/endAcceleration, startJerk/endJerk, startSnap/endSnap), implicitly matching only the degree-9 default formulation. It does **not** yet generalize across degree 3/5/7, and it does **not** yet encode the necessary/possible/redundant classification introduced in this document.

The classification introduced here is the theoretical basis for a future revision of that structure, for example along these lines:

$$\text{BoundaryConditions} = \{\text{necessaryConditions}[], \text{possibleConditions}[], \text{redundantConditions}[]\}$$

where such a revised structure would hold:

- necessaryConditions[]: the conditions required by the chosen degree and formulation;
- possibleConditions[]: conditions usable in alternative formulations;
- redundantConditions[]: conditions that exceed the default fixed formulation.

Reconciling the current fixed-field structure with this degree-parametrized, classified structure is an open point, to be resolved during PolynomialCoefficientDetermination or the later software design phase – not something already implemented today.

In later implementation, each condition will likely need a more explicit internal representation, for example:

- derivative order;
- endpoint identifier;
- symbolic or numeric value.

## Worked Symbolic Considerations

### Example 1: Cubic Segment

For a cubic segment with fixed duration, the unknown coefficients are:

$$a_0, a_1, a_2, a_3$$

The natural endpoint conditions are:

$$x_0, x_f, v_0, v_f$$

This gives four independent scalar equations for four unknowns.

If one now adds  $a_0$  and  $a_f$  as prescribed endpoint accelerations without changing the formulation, the cubic problem becomes overconstrained.

### Example 2: Quintic Segment

For a quintic segment, the unknown coefficients are:

$$a_0, a_1, a_2, a_3, a_4, a_5$$

The natural endpoint conditions are:

$$x_0, x_f, v_0, v_f, a_0, a_f$$

This is why quintic polynomials are often associated with endpoint control up to acceleration.

### Example 3: Ninth-Degree Segment

For a ninth-degree segment, the natural symmetric endpoint set is:

$$\mathcal{S}_0 = (x_0, v_0, a_0, j_0, s_0), \quad \mathcal{S}_f = (x_f, v_f, a_f, j_f, s_f)$$

This gives ten scalar conditions, matching the ten coefficients of the polynomial exactly.

That exact structural match is one of the main reasons why the degree-nine case is the central reference of the project.

## Open Questions

- In later implementation, should a boundary condition be stored primarily by derivative order or by a more descriptive enum-based type?
- When alternative formulations are introduced, should the project keep the same necessary/possible/redundant naming or add a fourth category for “active in the current variant”?
- For 3D trajectories, should the user interface expose boundary conditions axis by axis, or primarily in vector form?
- Should 02-data-model.md’s BoundaryConditions structure be revised to the degree-parametrized, classified form sketched in *Relation To The Data Model*, and if so, at which point in the roadmap should that revision happen?

## Related Documents

- SymbolsAndNomenclature.md
- NinthDegreePolynomialTheory.md
- PolynomialCoefficientDetermination
- 02-data-model.md